

Check databases inside container:

```
docker exec -it test_erp_db_pgsql psql -U postgres
```

Then inside PostgreSQL:

```
\l
```

Exit:

```
\q
```

Now restore your `erp_db.sql` dump.

If file is on host machine at:

```
/home/aayush/Desktop/erp_db.sql
```

Run:

```
docker exec -i test_erp_db_pgsql psql -U postgres < /home/aayush/Desktop/erp_db.sql
```

If dump expects database `erp_db`, first create it:

```
docker exec -it test_erp_db_pgsql psql -U postgres
```

Then:

```
CREATE DATABASE erp_db;  
\q
```

Then restore specifically into that DB:

```
docker exec -i test_erp_db_pgsql psql -U postgres -d erp_db < /home/aayush/Desktop/erp_db.sql
```

□

```
root@aayush:/home/aayush# docker exec -it test_erp_db_pgsql -U postgres OCI runtime exec failed: exec failed: unable to start container process: exec: "-U": executable file not found in $PATH: unknown
```

You missed `psql` command.

Correct command:

```
docker exec -it test_erp_db_pgsql psql -U postgres
```

Then:

\l



how to open the list of tables inside the database:

First connect to database:

```
docker exec -it test_erp_db_pgsql psql -U postgres -d erp_db
```

Then list tables:

```
\dt
```

List all schemas + tables:

```
\dt *.*
```

commands :

how to stop the docker currently running

Stop the container:

```
docker stop test_erp_db_pgsql
```

Check status:

```
docker ps
```

1. Create Dockerfile inside:

```
/home/aayush/Desktop/ERP_College_Django/ExecutableCodes/erpCollege
```

Content:

```
FROM python:3.13-slim
```

```
WORKDIR /app
```

```
COPY . /app
```

```
RUN pip install --no-cache-dir -r pip_dependency.txt
```

```
EXPOSE 8000
```

```
CMD ["python", "manage.py", "runserver", "0.0.0.0:8000"]
```

2. Your `pip_dependency.txt` should contain:

```
Django==6.0.3
psycopg2-binary==2.9.12
asgiref==3.11.1
sqlparse==0.5.5
tzdata==2025.3
```

3. Build image:

```
cd /home/aayush/Desktop/ERP_College_Django/ExecutableCodes/erpCollege
docker build -t erpcollege_app .
```

4. Check postgres container name:

```
docker ps
```

Example:

```
test_erp_db_pgsql
```

5. Create docker network:

```
docker network create erp_network
```

6. Connect postgres container to network:

```
docker network connect erp_network test_erp_db_pgsql
```

7. Run Django container:

```
docker run -d \
--name erpcollege_container \
-p 8000:8000 \
--network erp_network \
erpcollege_app
```

8. In Django `settings.py`, update DATABASES:

```
DATABASES = {
    'default': {
        'ENGINE': 'django.db.backends.postgresql',
        'NAME': 'erp_db',
```

```
        'USER': 'postgres',  
        'PASSWORD': 'root',  
        'HOST': 'test_erp_db_pgsql',  
        'PORT': '5432',  
    }  
}
```

IMPORTANT:

HOST = postgres container name

NOT localhost.

9. Restart Django container:

```
docker restart erpcollege_container
```

10. Open:

```
http://localhost:8000
```

11. To see logs:

```
docker logs -f erpcollege_container
```

12. To enter container:

```
docker exec -it erpcollege_container bash
```

13. To verify DB connection inside container:

```
python manage.py dbshell
```

If connected, your Django container is successfully talking to PostgreSQL container.

You are inside:

```
/home/aayush/Desktop/ERP_College_Django/ExecutableCodes/erpCollege/erpCollege
```

But your Dockerfile should be in:

/home/aayush/Desktop/ERP_College_Django/ExecutableCodes/erpCollege

Go back one folder:

```
cd ..
```

Now check:

```
ls
```

You should see:

```
Dockerfile
manage.py
appErpAdmin
appFaculty
...
```

If Dockerfile not present, create it:

```
nano Dockerfile
```

Paste:

```
FROM python:3.13-slim

WORKDIR /app

COPY . /app

RUN pip install --no-cache-dir -r pip_dependency.txt

EXPOSE 8000

CMD ["python", "manage.py", "runserver", "0.0.0.0:8000"]
```

Save:

```
CTRL + O
ENTER
CTRL + X
```

Then run:

```
docker build -t erpcollege_app .
```

erpcollege_app is the IMAGE name.

erpcollege_container is the CONTAINER name.

So stop using:

```
docker stop erpcollege_app
```

Use:

```
docker stop erpcollege_container
```

Now run postgres container again.

First see all containers including stopped:

```
docker ps -a
```

Find postgres container name. Example:

```
test_erp_db_pgsql
```

Start it:

```
docker start test_erp_db_pgsql
```

Now check:

```
docker ps
```

You should see BOTH:

```
erpcollege_container  
test_erp_db_pgsql
```

If postgres container was deleted completely, create again:

```
docker run -d \  
--name test_erp_db_pgsql \  
-e POSTGRES_PASSWORD=root \  
-e POSTGRES_DB=erp_db \  
-p 5432:5432 \  
postgres
```

Then import SQL backup:

```
docker exec -i test_erp_db_pgsql psql -U postgres -d erp_db < erp_db.sql
```

Then restart Django container:

```
docker restart erpcollege_container
```

Error means:

Django container cannot find postgres container hostname

This happens because BOTH containers are NOT inside same docker network.

Fix step-by-step:

1. Check postgres container exists:

```
docker ps -a
```

Suppose postgres container name is:

```
test_erp_db_pgsql
```

2. Create network:

```
docker network create erp_network
```

If already exists, ignore.

3. Connect postgres container to network:

```
docker network connect erp_network test_erp_db_pgsql
```

4. Stop and remove django container:

```
docker stop erpcollege_container
```

```
docker rm erpcollege_container
```

5. Run django container AGAIN with same network:

```
docker run -d \  
--name erpcollege_container \  
-p 8000:8000 \  
--network erp_network \  
erpcollege_app
```

6. Verify both are in same network:

```
docker network inspect erp_network
```

You should see BOTH:

```
test_erp_db_pgsql  
erpcollege_container
```

7. Your Django DB settings should be:

```
'HOST': 'test_erp_db_pgsql',
```

NOT localhost.

Now open:

```
http://localhost:8000
```

It should connect successfully.

```
Django container cannot resolve/find postgres container name:  
test_erp_db_pgsql
```

Meaning:

Both containers are NOT connected to same docker network

Run EXACTLY these commands now:

1.

```
docker network create erp_network
```

2.

```
docker network connect erp_network test_erp_db_pgsql
```

3.

```
docker stop erpcollege_container
```

4.

```
docker rm erpcollege_container
```

5.

```
docker run -d \  
--name erpcollege_container \  
-p 8000:8000 \  
--network erp_network \  
erpcollege_app
```

6. Verify:

```
docker network inspect erp_network
```

Inside "Containers" section BOTH must appear:

```
test_erp_db_pgsql  
erpcollege_container
```

If postgres container missing there → issue not fixed.

7. Then check logs:

```
docker logs -f erpcollege_container
```

You should NOT see:

```
could not translate host name
```

anymore.